# Lecture 4: Sequence Re-assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATG

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

*Digital trees, tries, suffix trees*

# Outline

1. **Reconstruction**
2. Digital Trees
3. Suffix Trees
4. Repeats

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGT

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# DNA Sequencing

- In modern whole shotgun genome sequencing the full genome is broken into small pieces
- The pieces are then read by a sequencing machine
- This reads short sections (100–1000) bases
- The **reads** are then assembled to construct the full genome

# DNA Sequencing

- In modern whole shotgun genome sequencing the full genome is broken into small pieces
- The pieces are then read by a sequencing machine
- This reads short sections (100–1000) bases
- The **reads** are then assembled to construct the full genome

# DNA Sequencing

- In modern whole shotgun genome sequencing the full genome is broken into small pieces
- The pieces are then read by a sequencing machine
- This reads short sections (100–1000) bases
- The **reads** are then assembled to construct the full genome

# DNA Sequencing

- In modern whole shotgun genome sequencing the full genome is broken into small pieces
- The pieces are then read by a sequencing machine
- This reads short sections (100–1000) bases
- The **reads** are then assembled to construct the full genome

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTA

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAA

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTA

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACT

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTC

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCG

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGC

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCAT

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATG

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGT

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGTCC

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGTCCCG

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGTCCCGTC

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGTCCCGTCT

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGTCCCGTCTCG

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGTCCCGTCTCGA

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGTCCCGTCTCGAGC

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGTCCCGTCTCGAGCA

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGTCCCGTCTCGAGCAA

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Sequencing and Assembly

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGTCCCGTCTCGAGCAACG

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Reads and Contigs

- In the language of DNA reconstruction we are given a set of **reads**
- Our job is to reconstruct them into **contigs**
- The problem is complicated because there are errors in the reads (0.1-2%)
- Shotgun was important in speeding up the human genome project (there is some argument about how important)
- What data structure should we use?

# Reads and Contigs

- In the language of DNA reconstruction we are given a set of **reads**
- Our job is to reconstruct them into **contigs**
- The problem is complicated because there are errors in the reads (0.1-2%)
- Shotgun was important in speeding up the human genome project (there is some argument about how important)
- What data structure should we use?

# Reads and Contigs

- In the language of DNA reconstruction we are given a set of **reads**
- Our job is to reconstruct them into **contigs**
- The problem is complicated because there are errors in the reads (0.1-2%)
- Shotgun was important in speeding up the human genome project (there is some argument about how important)
- What data structure should we use?

# Reads and Contigs

- In the language of DNA reconstruction we are given a set of **reads**
- Our job is to reconstruct them into **contigs**
- The problem is complicated because there are errors in the reads (0.1-2%)
- Shotgun was important in speeding up the human genome project (there is some argument about how important)
- What data structure should we use?

# Reads and Contigs

- In the language of DNA reconstruction we are given a set of **reads**
- Our job is to reconstruct them into **contigs**
- The problem is complicated because there are errors in the reads (0.1-2%)
- Shotgun was important in speeding up the human genome project (there is some argument about how important)
- What data structure should we use?

# Outline

1. Reconstruction
2. **Digital Trees**
3. Suffix Trees
4. Repeats

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGT

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Digital Trees or Tries

- A **Trie** (pron. 'try') or **digital tree** is a multiway tree often used for storing large sets of words
- They are trees with a possible branch for every letter of an alphabet
- Their names comes from the word $re$**trie**$val$
- Tries usually compactify the edges in the tree
- All words end with a special letter "$"

# Digital Trees or Tries

- A **Trie** (pron. 'try') or **digital tree** is a multiway tree often used for storing large sets of words
- They are trees with a possible branch for every letter of an alphabet
- Their names comes from the word $re$**trie**$val$
- Tries usually compactify the edges in the tree
- All words end with a special letter "$\$$"

# Digital Trees or Tries

- A **Trie** (pron. 'try') or **digital tree** is a multiway tree often used for storing large sets of words
- They are trees with a possible branch for every letter of an alphabet
- Their names comes from the word *re***trie***val*
- Tries usually compactify the edges in the tree
- All words end with a special letter "$"

# Digital Trees or Tries

- A **Trie** (pron. 'try') or **digital tree** is a multiway tree often used for storing large sets of words
- They are trees with a possible branch for every letter of an alphabet
- Their names comes from the word $re$**trie**$val$
- Tries usually compactify the edges in the tree
- All words end with a special letter "$"

# Digital Trees or Tries

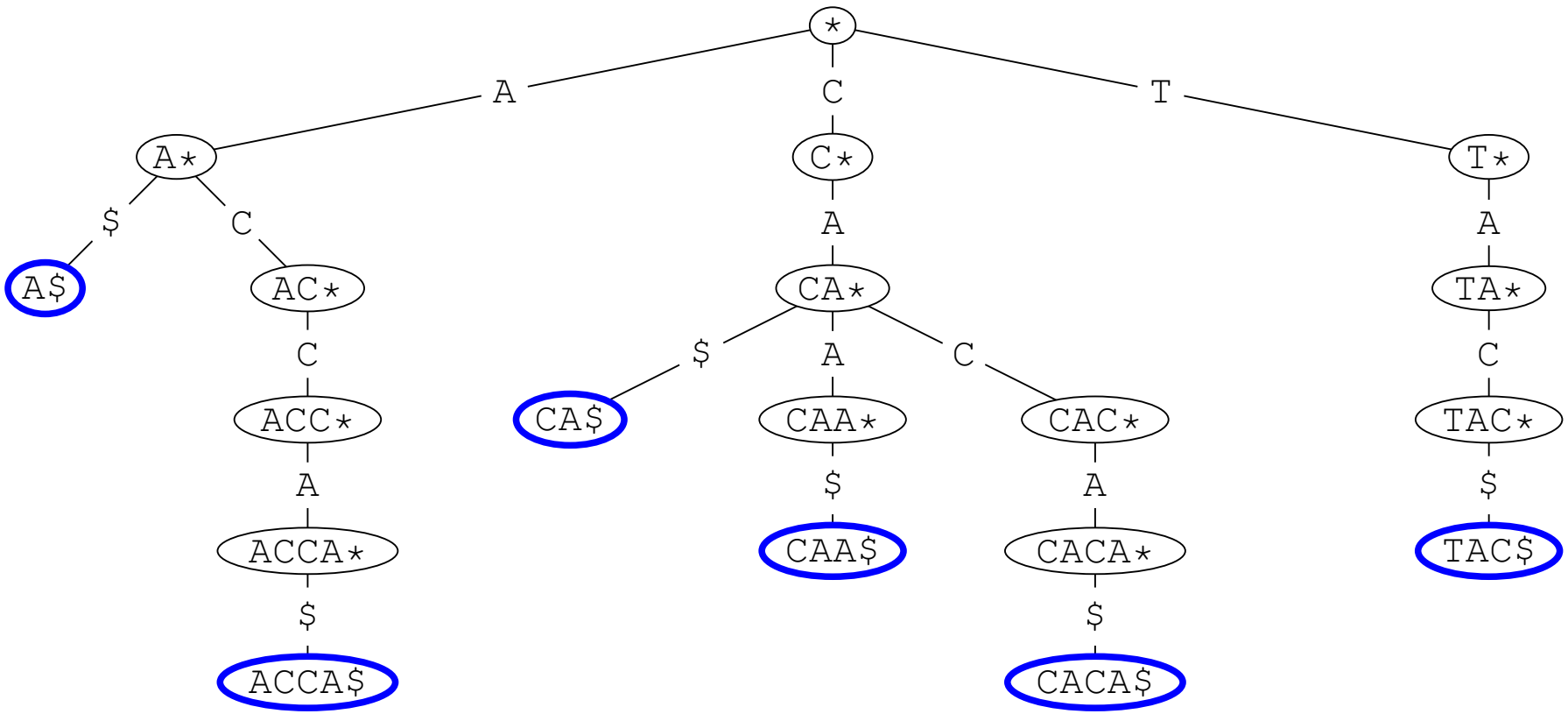
- A **Trie** (pron. 'try') or **digital tree** is a multiway tree often used for storing large sets of words
- They are trees with a possible branch for every letter of an alphabet
- Their names comes from the word $re$**trie**$val$
- Tries usually compactify the edges in the tree
- All words end with a special letter "$"

# Trie

{A\$, ACCA\$, CA\$, CAA\$, CACA\$, TAC\$}

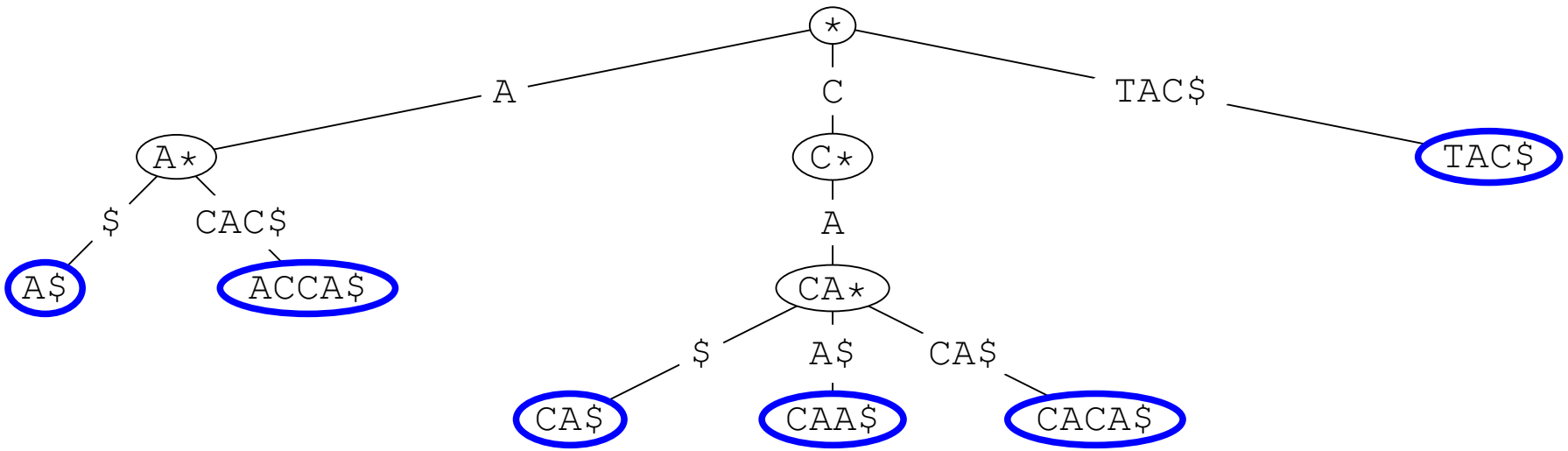
# Trie

{A$, ACCA$, CA$, CAA$, CACA$, TAC$}



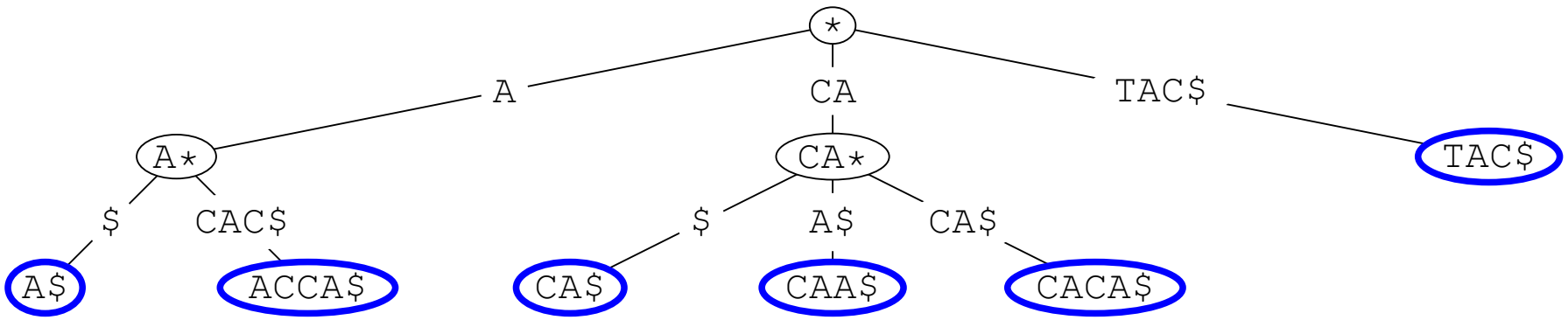
# Trie

{A, ACCA, CA, CAA, CACA, TAC}



# Trie

{A, ACCA, CA, CAA, CACA, TAC}



# General Uses of Tries

- Tries are yet another way of implementing sets
- They provide quick insertion, deletion and find
- Typically considerably quicker than binary trees and hash tables
- They are particularly good for spell checkers, completion algorithms, longest-prefix matching, hyphenation
- Each search finds the longest match between the words in the set and the query

# General Uses of Tries

- Tries are yet another way of implementing sets
- They provide quick insertion, deletion and find
- Typically considerably quicker than binary trees and hash tables
- They are particularly good for spell checkers, completion algorithms, longest-prefix matching, hyphenation
- Each search finds the longest match between the words in the set and the query

# General Uses of Tries

- Tries are yet another way of implementing sets
- They provide quick insertion, deletion and find
- Typically considerably quicker than binary trees and hash tables
- They are particularly good for spell checkers, completion algorithms, longest-prefix matching, hyphenation
- Each search finds the longest match between the words in the set and the query

# General Uses of Tries

- Tries are yet another way of implementing sets
- They provide quick insertion, deletion and find
- Typically considerably quicker than binary trees and hash tables
- They are particularly good for spell checkers, completion algorithms, longest-prefix matching, hyphenation
- Each search finds the longest match between the words in the set and the query

# General Uses of Tries

- Tries are yet another way of implementing sets
- They provide quick insertion, deletion and find
- Typically considerably quicker than binary trees and hash tables
- They are particularly good for spell checkers, completion algorithms, longest-prefix matching, hyphenation
- Each search finds the longest match between the words in the set and the query

# Tries for Sequence Analysis

- Tries can be used to store DNA or proteins
- For reconstruction of DNA tries are very useful for finding the next fitting section
- They are useful when there are errors, as by searching a small sub-tree you can find a good match (same reason they are good for completion and spell checking)
- For DNA you have a four way (5 way?) tree, which means your tree is deep but it doesn't waste so much memory

# Tries for Sequence Analysis

- Tries can be used to store DNA or proteins
- For reconstruction of DNA tries are very useful for finding the next fitting section
- They are useful when there are errors, as by searching a small sub-tree you can find a good match (same reason they are good for completion and spell checking)
- For DNA you have a four way (5 way?) tree, which means your tree is deep but it doesn't waste so much memory

# Tries for Sequence Analysis

- Tries can be used to store DNA or proteins
- For reconstruction of DNA tries are very useful for finding the next fitting section
- They are useful when there are errors, as by searching a small sub-tree you can find a good match (same reason they are good for completion and spell checking)
- For DNA you have a four way (5 way?) tree, which means your tree is deep but it doesn't waste so much memory

# Tries for Sequence Analysis

- Tries can be used to store DNA or proteins
- For reconstruction of DNA tries are very useful for finding the next fitting section
- They are useful when there are errors, as by searching a small sub-tree you can find a good match (same reason they are good for completion and spell checking)
- For DNA you have a four way (5 way?) tree, which means your tree is deep but it doesn't waste so much memory

# Trie for 31 Most Common English Words

| |
|---|
| |
| $ |
| A |
| B |
| C |
| D |
| E |
| F |
| G |
| H |
| I |
| J |
| K |
| L |
| M |
| N |
| O |
| P |
| Q |
| R |
| S |
| T |
| U |
| V |
| W |
| X |
| Y |
| Z |

# Trie for 31 Most Common English Words

```
add("THAT")
```

| |
|---|
| |
| $ |
| A |
| B |
| C |
| D |
| E |
| F |
| G |
| H |
| I |
| J |
| K |
| L |
| M |
| N |
| O |
| P |
| Q |
| R |
| S |
| T |
| U |
| V |
| W |
| X |
| Y |
| Z |

# Trie for 31 Most Common English Words

```
add("THAT")
```

| | 0 |
|---|---|
| $ | |
| A | |
| B | |
| C | |
| D | |
| E | |
| F | |
| G | |
| H | |
| I | |
| J | |
| K | |
| L | |
| M | |
| N | |
| O | |
| P | |
| Q | |
| R | |
| S | |
| T | THAT$ |
| U | |
| V | |
| W | |
| X | |
| Y | |
| Z | |

# Trie for 31 Most Common English Words

```
add("IN")
```

| | 0 |
|---|---|
| $ | |
| A | |
| B | |
| C | |
| D | |
| E | |
| F | |
| G | |
| H | |
| I | |
| J | |
| K | |
| L | |
| M | |
| N | |
| O | |
| P | |
| Q | |
| R | |
| S | |
| T | THAT$ |
| U | |
| V | |
| W | |
| X | |
| Y | |
| Z | |

# Trie for 31 Most Common English Words

```
add("IN")
```

| | 0 |
|---|---|
| $ | |
| A | |
| B | |
| C | |
| D | |
| E | |
| F | |
| G | |
| H | |
| I | IN$ |
| J | |
| K | |
| L | |
| M | |
| N | |
| O | |
| P | |
| Q | |
| R | |
| S | |
| T | THAT$ |
| U | |
| V | |
| W | |
| X | |
| Y | |
| Z | |

# Trie for 31 Most Common English Words

```
add("IT")
```

| | 0 |
|---|---|
| $ | |
| A | |
| B | |
| C | |
| D | |
| E | |
| F | |
| G | |
| H | |
| I | IN$ |
| J | |
| K | |
| L | |
| M | |
| N | |
| O | |
| P | |
| Q | |
| R | |
| S | |
| T | THAT$ |
| U | |
| V | |
| W | |
| X | |
| Y | |
| Z | |

# Trie for 31 Most Common English Words

```
add("IT")
```

| | 0 | 1 |
|---|---|---|
| $ | | |
| A | | |
| B | | |
| C | | |
| D | | |
| E | | |
| F | | |
| G | | |
| H | | |
| I | I* (1) | |
| J | | |
| K | | |
| L | | |
| M | | |
| N | | IN$ |
| O | | |
| P | | |
| Q | | |
| R | | |
| S | | |
| T | THAT$ | IT$ |
| U | | |
| V | | |
| W | | |
| X | | |
| Y | | |
| Z | | |

# Trie for 31 Most Common English Words

```
add("I")
```

| | 0 | 1 |
|---|---|---|
| $ | | |
| A | | |
| B | | |
| C | | |
| D | | |
| E | | |
| F | | |
| G | | |
| H | | |
| I | I* (1) | |
| J | | |
| K | | |
| L | | |
| M | | |
| N | | IN$ |
| O | | |
| P | | |
| Q | | |
| R | | |
| S | | |
| T | THAT$ | IT$ |
| U | | |
| V | | |
| W | | |
| X | | |
| Y | | |
| Z | | |

# Trie for 31 Most Common English Words

```
add("I")
```

| | 0 | 1 |
|---|---|---|
| $ | | I$ |
| A | | |
| B | | |
| C | | |
| D | | |
| E | | |
| F | | |
| G | | |
| H | | |
| I | I* (1) | |
| J | | |
| K | | |
| L | | |
| M | | |
| N | | IN$ |
| O | | |
| P | | |
| Q | | |
| R | | |
| S | | |
| T | THAT$ | IT$ |
| U | | |
| V | | |
| W | | |
| X | | |
| Y | | |
| Z | | |

# Trie for 31 Most Common English Words

```
add("HAD")
```

| | 0 | 1 |
|---|---|---|
| $ | | I$ |
| A | | |
| B | | |
| C | | |
| D | | |
| E | | |
| F | | |
| G | | |
| H | | |
| I | I* (1) | |
| J | | |
| K | | |
| L | | |
| M | | |
| N | | IN$ |
| O | | |
| P | | |
| Q | | |
| R | | |
| S | | |
| T | THAT$ | IT$ |
| U | | |
| V | | |
| W | | |
| X | | |
| Y | | |
| Z | | |

# Trie for 31 Most Common English Words

```
add("HAD")
```

| | 0 | 1 |
|---|---|---|
| $ | | I$ |
| A | | |
| B | | |
| C | | |
| D | | |
| E | | |
| F | | |
| G | | |
| H | HAD$ | |
| I | I* (1) | |
| J | | |
| K | | |
| L | | |
| M | | |
| N | | IN$ |
| O | | |
| P | | |
| Q | | |
| R | | |
| S | | |
| T | THAT$ | IT$ |
| U | | |
| V | | |
| W | | |
| X | | |
| Y | | |
| Z | | |

# Trie for 31 Most Common English Words

```
add("NOT")
```

| | 0 | 1 |
|---|---|---|
| $ | | I$ |
| A | | |
| B | | |
| C | | |
| D | | |
| E | | |
| F | | |
| G | | |
| H | HAD$ | |
| I | I* (1) | |
| J | | |
| K | | |
| L | | |
| M | | |
| N | | IN$ |
| O | | |
| P | | |
| Q | | |
| R | | |
| S | | |
| T | THAT$ | IT$ |
| U | | |
| V | | |
| W | | |
| X | | |
| Y | | |
| Z | | |

# Trie for 31 Most Common English Words

```
add("NOT")
```

| | 0 | 1 |
|---|---|---|
| $ | | I$ |
| A | | |
| B | | |
| C | | |
| D | | |
| E | | |
| F | | |
| G | | |
| H | HAD$ | |
| I | I* (1) | |
| J | | |
| K | | |
| L | | |
| M | | |
| N | NOT$ | IN$ |
| O | | |
| P | | |
| Q | | |
| R | | |
| S | | |
| T | THAT$ | IT$ |
| U | | |
| V | | |
| W | | |
| X | | |
| Y | | |
| Z | | |

# Trie for 31 Most Common English Words

```
add("WITH")
```

| | 0 | 1 |
|---|---|---|
| $ | | I$ |
| A | | |
| B | | |
| C | | |
| D | | |
| E | | |
| F | | |
| G | | |
| H | HAD$ | |
| I | I* (1) | |
| J | | |
| K | | |
| L | | |
| M | | |
| N | NOT$ | IN$ |
| O | | |
| P | | |
| Q | | |
| R | | |
| S | | |
| T | THAT$ | IT$ |
| U | | |
| V | | |
| W | | |
| X | | |
| Y | | |
| Z | | |

# Trie for 31 Most Common English Words

```
add("WITH")
```

| | 0 | 1 |
|---|---|---|
| $ | | I$ |
| A | | |
| B | | |
| C | | |
| D | | |
| E | | |
| F | | |
| G | | |
| H | HAD$ | |
| I | I* (1) | |
| J | | |
| K | | |
| L | | |
| M | | |
| N | NOT$ | IN$ |
| O | | |
| P | | |
| Q | | |
| R | | |
| S | | |
| T | THAT$ | IT$ |
| U | | |
| V | | |
| W | WITH$ | |
| X | | |
| Y | | |
| Z | | |

# Trie for 31 Most Common English Words

```
add("HIS")
```

| | 0 | 1 |
|---|---|---|
| $ | | I$ |
| A | | |
| B | | |
| C | | |
| D | | |
| E | | |
| F | | |
| G | | |
| H | HAD$ | |
| I | I* (1) | |
| J | | |
| K | | |
| L | | |
| M | | |
| N | NOT$ | IN$ |
| O | | |
| P | | |
| Q | | |
| R | | |
| S | | |
| T | THAT$ | IT$ |
| U | | |
| V | | |
| W | WITH$ | |
| X | | |
| Y | | |
| Z | | |

# Trie for 31 Most Common English Words

```
add("HIS")
```

| | 0 | 1 | 2 |
|---|---|---|---|
| $ | | I$ | |
| A | | | HAD$ |
| B | | | |
| C | | | |
| D | | | |
| E | | | |
| F | | | |
| G | | | |
| H | H* (2) | | |
| I | I* (1) | | HIS$ |
| J | | | |
| K | | | |
| L | | | |
| M | | | |
| N | NOT$ | IN$ | |
| O | | | |
| P | | | |
| Q | | | |
| R | | | |
| S | | | |
| T | THAT$ | IT$ | |
| U | | | |
| V | | | |
| W | WITH$ | | |
| X | | | |
| Y | | | |
| Z | | | |

# Trie for 31 Most Common English Words

```
add("HAVE")
```

| | 0 | 1 | 2 |
|---|---|---|---|
| $ | | I$ | |
| A | | | HAD$ |
| B | | | |
| C | | | |
| D | | | |
| E | | | |
| F | | | |
| G | | | |
| H | H* (2) | | |
| I | I* (1) | | HIS$ |
| J | | | |
| K | | | |
| L | | | |
| M | | | |
| N | NOT$ | IN$ | |
| O | | | |
| P | | | |
| Q | | | |
| R | | | |
| S | | | |
| T | THAT$ | IT$ | |
| U | | | |
| V | | | |
| W | WITH$ | | |
| X | | | |
| Y | | | |
| Z | | | |

# Trie for 31 Most Common English Words

```
add("HAVE")
```

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| $ | | I$ | | |
| A | | | HA* (3) | |
| B | | | | |
| C | | | | |
| D | | | | HAD$ |
| E | | | | |
| F | | | | |
| G | | | | |
| H | H* (2) | | | |
| I | I* (1) | | HIS$ | |
| J | | | | |
| K | | | | |
| L | | | | |
| M | | | | |
| N | NOT$ | IN$ | | |
| O | | | | |
| P | | | | |
| Q | | | | |
| R | | | | |
| S | | | | |
| T | THAT$ | IT$ | | |
| U | | | | |
| V | | | | HAVE$ |
| W | WITH$ | | | |
| X | | | | |
| Y | | | | |
| Z | | | | |

# Trie for 31 Most Common English Words

```
add("A")
```

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| $ | | I$ | | |
| A | | | HA* (3) | |
| B | | | | |
| C | | | | |
| D | | | | HAD$ |
| E | | | | |
| F | | | | |
| G | | | | |
| H | H* (2) | | | |
| I | I* (1) | | HIS$ | |
| J | | | | |
| K | | | | |
| L | | | | |
| M | | | | |
| N | NOT$ | IN$ | | |
| O | | | | |
| P | | | | |
| Q | | | | |
| R | | | | |
| S | | | | |
| T | THAT$ | IT$ | | |
| U | | | | |
| V | | | | HAVE$ |
| W | WITH$ | | | |
| X | | | | |
| Y | | | | |
| Z | | | | |

# Trie for 31 Most Common English Words

```
add("A")
```

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| $ | | I$ | | |
| A | A$ | | HA* (3) | |
| B | | | | |
| C | | | | |
| D | | | | HAD$ |
| E | | | | |
| F | | | | |
| G | | | | |
| H | H* (2) | | | |
| I | I* (1) | | HIS$ | |
| J | | | | |
| K | | | | |
| L | | | | |
| M | | | | |
| N | NOT$ | IN$ | | |
| O | | | | |
| P | | | | |
| Q | | | | |
| R | | | | |
| S | | | | |
| T | THAT$ | IT$ | | |
| U | | | | |
| V | | | | HAVE$ |
| W | WITH$ | | | |
| X | | | | |
| Y | | | | |
| Z | | | | |

# Trie for 31 Most Common English Words

```
add("WHICH")
```

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| $ | | I$ | | |
| A | A$ | | HA* (3) | |
| B | | | | |
| C | | | | |
| D | | | | HAD$ |
| E | | | | |
| F | | | | |
| G | | | | |
| H | H* (2) | | | |
| I | I* (1) | | HIS$ | |
| J | | | | |
| K | | | | |
| L | | | | |
| M | | | | |
| N | NOT$ | IN$ | | |
| O | | | | |
| P | | | | |
| Q | | | | |
| R | | | | |
| S | | | | |
| T | THAT$ | IT$ | | |
| U | | | | |
| V | | | | HAVE$ |
| W | WITH$ | | | |
| X | | | | |
| Y | | | | |
| Z | | | | |

# Trie for 31 Most Common English Words

```
add("WHICH")
```

| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $ | | I$ | | | |
| A | A$ | | HA* (3) | | |
| B | | | | | |
| C | | | | | |
| D | | | | HAD$ | |
| E | | | | | |
| F | | | | | |
| G | | | | | |
| H | H* (2) | | | | WHICH$ |
| I | I* (1) | | HIS$ | | WITH$ |
| J | | | | | |
| K | | | | | |
| L | | | | | |
| M | | | | | |
| N | NOT$ | IN$ | | | |
| O | | | | | |
| P | | | | | |
| Q | | | | | |
| R | | | | | |
| S | | | | | |
| T | THAT$ | IT$ | | | |
| U | | | | | |
| V | | | | HAVE$ | |
| W | W* (4) | | | | |
| X | | | | | |
| Y | | | | | |
| Z | | | | | |

# Trie for 31 Most Common English Words

```
add("HER")
```

| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $ | | I$ | | | |
| A | A$ | | HA* (3) | | |
| B | | | | | |
| C | | | | | |
| D | | | | HAD$ | |
| E | | | | | |
| F | | | | | |
| G | | | | | |
| H | H* (2) | | | | WHICH$ |
| I | I* (1) | | HIS$ | | WITH$ |
| J | | | | | |
| K | | | | | |
| L | | | | | |
| M | | | | | |
| N | NOT$ | IN$ | | | |
| O | | | | | |
| P | | | | | |
| Q | | | | | |
| R | | | | | |
| S | | | | | |
| T | THAT$ | IT$ | | | |
| U | | | | | |
| V | | | | HAVE$ | |
| W | W* (4) | | | | |
| X | | | | | |
| Y | | | | | |
| Z | | | | | |

# Trie for 31 Most Common English Words

```
add("HER")
```

| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $ | | I$ | | | |
| A | A$ | | HA* (3) | | |
| B | | | | | |
| C | | | | | |
| D | | | | HAD$ | |
| E | | | HER$ | | |
| F | | | | | |
| G | | | | | |
| H | H* (2) | | | | WHICH$ |
| I | I* (1) | | HIS$ | | WITH$ |
| J | | | | | |
| K | | | | | |
| L | | | | | |
| M | | | | | |
| N | NOT$ | IN$ | | | |
| O | | | | | |
| P | | | | | |
| Q | | | | | |
| R | | | | | |
| S | | | | | |
| T | THAT$ | IT$ | | | |
| U | | | | | |
| V | | | | HAVE$ | |
| W | W* (4) | | | | |
| X | | | | | |
| Y | | | | | |
| Z | | | | | |

# Trie for 31 Most Common English Words

```
add("AT")
```

| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $ | | I$ | | | |
| A | A$ | | HA* (3) | | |
| B | | | | | |
| C | | | | | |
| D | | | | HAD$ | |
| E | | | HER$ | | |
| F | | | | | |
| G | | | | | |
| H | H* (2) | | | | WHICH$ |
| I | I* (1) | | HIS$ | | WITH$ |
| J | | | | | |
| K | | | | | |
| L | | | | | |
| M | | | | | |
| N | NOT$ | IN$ | | | |
| O | | | | | |
| P | | | | | |
| Q | | | | | |
| R | | | | | |
| S | | | | | |
| T | THAT$ | IT$ | | | |
| U | | | | | |
| V | | | | HAVE$ | |
| W | W* (4) | | | | |
| X | | | | | |
| Y | | | | | |
| Z | | | | | |

# Trie for 31 Most Common English Words

```
add("AT")
```

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ |
| A | A* (5) | | HA* (3) | | | |
| B | | | | | | |
| C | | | | | | |
| D | | | | HAD$ | | |
| E | | | HER$ | | | |
| F | | | | | | |
| G | | | | | | |
| H | H* (2) | | | | WHICH$ | |
| I | I* (1) | | HIS$ | | WITH$ | |
| J | | | | | | |
| K | | | | | | |
| L | | | | | | |
| M | | | | | | |
| N | NOT$ | IN$ | | | | |
| O | | | | | | |
| P | | | | | | |
| Q | | | | | | |
| R | | | | | | |
| S | | | | | | |
| T | THAT$ | IT$ | | | | AT$ |
| U | | | | | | |
| V | | | | HAVE$ | | |
| W | W* (4) | | | | | |
| X | | | | | | |
| Y | | | | | | |
| Z | | | | | | |

# Trie for 31 Most Common English Words

```
add("IS")
```

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ |
| A | A* (5) | | HA* (3) | | | |
| B | | | | | | |
| C | | | | | | |
| D | | | | HAD$ | | |
| E | | | HER$ | | | |
| F | | | | | | |
| G | | | | | | |
| H | H* (2) | | | | WHICH$ | |
| I | I* (1) | | HIS$ | | WITH$ | |
| J | | | | | | |
| K | | | | | | |
| L | | | | | | |
| M | | | | | | |
| N | NOT$ | IN$ | | | | |
| O | | | | | | |
| P | | | | | | |
| Q | | | | | | |
| R | | | | | | |
| S | | | | | | |
| T | THAT$ | IT$ | | | | AT$ |
| U | | | | | | |
| V | | | | HAVE$ | | |
| W | W* (4) | | | | | |
| X | | | | | | |
| Y | | | | | | |
| Z | | | | | | |

# Trie for 31 Most Common English Words

```
add("IS")
```

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ |
| A | A* (5) | | HA* (3) | | | |
| B | | | | | | |
| C | | | | | | |
| D | | | | HAD$ | | |
| E | | | HER$ | | | |
| F | | | | | | |
| G | | | | | | |
| H | H* (2) | | | | WHICH$ | |
| I | I* (1) | | HIS$ | | WITH$ | |
| J | | | | | | |
| K | | | | | | |
| L | | | | | | |
| M | | | | | | |
| N | NOT$ | IN$ | | | | |
| O | | | | | | |
| P | | | | | | |
| Q | | | | | | |
| R | | | | | | |
| S | | IS$ | | | | |
| T | THAT$ | IT$ | | | | AT$ |
| U | | | | | | |
| V | | | | HAVE$ | | |
| W | W* (4) | | | | | |
| X | | | | | | |
| Y | | | | | | |
| Z | | | | | | |

# Trie for 31 Most Common English Words

```
add("AND")
```

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ |
| A | A* (5) | | HA* (3) | | | |
| B | | | | | | |
| C | | | | | | |
| D | | | | HAD$ | | |
| E | | | HER$ | | | |
| F | | | | | | |
| G | | | | | | |
| H | H* (2) | | | | WHICH$ | |
| I | I* (1) | | HIS$ | | WITH$ | |
| J | | | | | | |
| K | | | | | | |
| L | | | | | | |
| M | | | | | | |
| N | NOT$ | IN$ | | | | |
| O | | | | | | |
| P | | | | | | |
| Q | | | | | | |
| R | | | | | | |
| S | | IS$ | | | | |
| T | THAT$ | IT$ | | | | AT$ |
| U | | | | | | |
| V | | | | HAVE$ | | |
| W | W* (4) | | | | | |
| X | | | | | | |
| Y | | | | | | |
| Z | | | | | | |

# Trie for 31 Most Common English Words

```
add("AND")
```

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ |
| A | A* (5) | | HA* (3) | | | |
| B | | | | | | |
| C | | | | | | |
| D | | | | HAD$ | | |
| E | | | HER$ | | | |
| F | | | | | | |
| G | | | | | | |
| H | H* (2) | | | | WHICH$ | |
| I | I* (1) | | HIS$ | | WITH$ | |
| J | | | | | | |
| K | | | | | | |
| L | | | | | | |
| M | | | | | | |
| N | NOT$ | IN$ | | | | AND$ |
| O | | | | | | |
| P | | | | | | |
| Q | | | | | | |
| R | | | | | | |
| S | | IS$ | | | | |
| T | THAT$ | IT$ | | | | AT$ |
| U | | | | | | |
| V | | | | HAVE$ | | |
| W | W* (4) | | | | | |
| X | | | | | | |
| Y | | | | | | |
| Z | | | | | | |

# Trie for 31 Most Common English Words

```
add("BUT")
```

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ |
| A | A* (5) | | HA* (3) | | | |
| B | | | | | | |
| C | | | | | | |
| D | | | | HAD$ | | |
| E | | | HER$ | | | |
| F | | | | | | |
| G | | | | | | |
| H | H* (2) | | | | WHICH$ | |
| I | I* (1) | | HIS$ | | WITH$ | |
| J | | | | | | |
| K | | | | | | |
| L | | | | | | |
| M | | | | | | |
| N | NOT$ | IN$ | | | | AND$ |
| O | | | | | | |
| P | | | | | | |
| Q | | | | | | |
| R | | | | | | |
| S | | IS$ | | | | |
| T | THAT$ | IT$ | | | | AT$ |
| U | | | | | | |
| V | | | | HAVE$ | | |
| W | W* (4) | | | | | |
| X | | | | | | |
| Y | | | | | | |
| Z | | | | | | |

# Trie for 31 Most Common English Words

`add("BUT")`

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ |
| A | A* (5) | | HA* (3) | | | |
| B | BUT$ | | | | | |
| C | | | | | | |
| D | | | | HAD$ | | |
| E | | | HER$ | | | |
| F | | | | | | |
| G | | | | | | |
| H | H* (2) | | | | WHICH$ | |
| I | I* (1) | | HIS$ | | WITH$ | |
| J | | | | | | |
| K | | | | | | |
| L | | | | | | |
| M | | | | | | |
| N | NOT$ | IN$ | | | | AND$ |
| O | | | | | | |
| P | | | | | | |
| Q | | | | | | |
| R | | | | | | |
| S | | IS$ | | | | |
| T | THAT$ | IT$ | | | | AT$ |
| U | | | | | | |
| V | | | | HAVE$ | | |
| W | W* (4) | | | | | |
| X | | | | | | |
| Y | | | | | | |
| Z | | | | | | |

# Trie for 31 Most Common English Words

```
add("THE")
```

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ |
| A | A* (5) | | HA* (3) | | | |
| B | BUT$ | | | | | |
| C | | | | | | |
| D | | | | HAD$ | | |
| E | | | HER$ | | | |
| F | | | | | | |
| G | | | | | | |
| H | H* (2) | | | | WHICH$ | |
| I | I* (1) | | HIS$ | | WITH$ | |
| J | | | | | | |
| K | | | | | | |
| L | | | | | | |
| M | | | | | | |
| N | NOT$ | IN$ | | | | AND$ |
| O | | | | | | |
| P | | | | | | |
| Q | | | | | | |
| R | | | | | | |
| S | | IS$ | | | | |
| T | THAT$ | IT$ | | | | AT$ |
| U | | | | | | |
| V | | | | HAVE$ | | |
| W | W* (4) | | | | | |
| X | | | | | | |
| Y | | | | | | |
| Z | | | | | | |

# Trie for 31 Most Common English Words

`add("THE")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | |
| A | A* (5) | | HA* (3) | | | | THAT$ |
| B | BUT$ | | | | | | |
| C | | | | | | | |
| D | | | | HAD$ | | | |
| E | | | HER$ | | | | THE$ |
| F | | | | | | | |
| G | | | | | | | |
| H | H* (2) | | | | WHICH$ | | |
| I | I* (1) | | HIS$ | | WITH$ | | |
| J | | | | | | | |
| K | | | | | | | |
| L | | | | | | | |
| M | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | |
| O | | | | | | | |
| P | | | | | | | |
| Q | | | | | | | |
| R | | | | | | | |
| S | | IS$ | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | |
| U | | | | | | | |
| V | | | | HAVE$ | | | |
| W | W* (4) | | | | | | |
| X | | | | | | | |
| Y | | | | | | | |
| Z | | | | | | | |

# Trie for 31 Most Common English Words

`add("FOR")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | |
| A | A* (5) | | HA* (3) | | | | THAT$ |
| B | BUT$ | | | | | | |
| C | | | | | | | |
| D | | | | HAD$ | | | |
| E | | | HER$ | | | | THE$ |
| F | | | | | | | |
| G | | | | | | | |
| H | H* (2) | | | | WHICH$ | | |
| I | I* (1) | | HIS$ | | WITH$ | | |
| J | | | | | | | |
| K | | | | | | | |
| L | | | | | | | |
| M | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | |
| O | | | | | | | |
| P | | | | | | | |
| Q | | | | | | | |
| R | | | | | | | |
| S | | IS$ | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | |
| U | | | | | | | |
| V | | | | HAVE$ | | | |
| W | W* (4) | | | | | | |
| X | | | | | | | |
| Y | | | | | | | |
| Z | | | | | | | |

# Trie for 31 Most Common English Words

`add("FOR")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | |
| A | A* (5) | | HA* (3) | | | | THAT$ |
| B | BUT$ | | | | | | |
| C | | | | | | | |
| D | | | | HAD$ | | | |
| E | | | HER$ | | | | THE$ |
| F | FOR$ | | | | | | |
| G | | | | | | | |
| H | H* (2) | | | | WHICH$ | | |
| I | I* (1) | | HIS$ | | WITH$ | | |
| J | | | | | | | |
| K | | | | | | | |
| L | | | | | | | |
| M | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | |
| O | | | | | | | |
| P | | | | | | | |
| Q | | | | | | | |
| R | | | | | | | |
| S | | IS$ | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | |
| U | | | | | | | |
| V | | | | HAVE$ | | | |
| W | W* (4) | | | | | | |
| X | | | | | | | |
| Y | | | | | | | |
| Z | | | | | | | |

# Trie for 31 Most Common English Words

```
add("ON")
```

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | |
| A | A* (5) | | HA* (3) | | | | THAT$ |
| B | BUT$ | | | | | | |
| C | | | | | | | |
| D | | | | HAD$ | | | |
| E | | | HER$ | | | | THE$ |
| F | FOR$ | | | | | | |
| G | | | | | | | |
| H | H* (2) | | | | WHICH$ | | |
| I | I* (1) | | HIS$ | | WITH$ | | |
| J | | | | | | | |
| K | | | | | | | |
| L | | | | | | | |
| M | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | |
| O | | | | | | | |
| P | | | | | | | |
| Q | | | | | | | |
| R | | | | | | | |
| S | | IS$ | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | |
| U | | | | | | | |
| V | | | | HAVE$ | | | |
| W | W* (4) | | | | | | |
| X | | | | | | | |
| Y | | | | | | | |
| Z | | | | | | | |

# Trie for 31 Most Common English Words

```
add("ON")
```

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | |
| A | A* (5) | | HA* (3) | | | | THAT$ |
| B | BUT$ | | | | | | |
| C | | | | | | | |
| D | | | | HAD$ | | | |
| E | | | HER$ | | | | THE$ |
| F | FOR$ | | | | | | |
| G | | | | | | | |
| H | H* (2) | | | | WHICH$ | | |
| I | I* (1) | | HIS$ | | WITH$ | | |
| J | | | | | | | |
| K | | | | | | | |
| L | | | | | | | |
| M | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | |
| O | ON$ | | | | | | |
| P | | | | | | | |
| Q | | | | | | | |
| R | | | | | | | |
| S | | IS$ | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | |
| U | | | | | | | |
| V | | | | HAVE$ | | | |
| W | W* (4) | | | | | | |
| X | | | | | | | |
| Y | | | | | | | |
| Z | | | | | | | |

# Trie for 31 Most Common English Words

```
add("HE")
```

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | |
| A | A* (5) | | HA* (3) | | | | THAT$ |
| B | BUT$ | | | | | | |
| C | | | | | | | |
| D | | | | HAD$ | | | |
| E | | | HER$ | | | | THE$ |
| F | FOR$ | | | | | | |
| G | | | | | | | |
| H | H* (2) | | | | WHICH$ | | |
| I | I* (1) | | HIS$ | | WITH$ | | |
| J | | | | | | | |
| K | | | | | | | |
| L | | | | | | | |
| M | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | |
| O | ON$ | | | | | | |
| P | | | | | | | |
| Q | | | | | | | |
| R | | | | | | | |
| S | | IS$ | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | |
| U | | | | | | | |
| V | | | | HAVE$ | | | |
| W | W* (4) | | | | | | |
| X | | | | | | | |
| Y | | | | | | | |
| Z | | | | | | | |

# Trie for 31 Most Common English Words

`add("HE")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ |
| A | A* (5) | | HA* (3) | | | | THAT$ | |
| B | BUT$ | | | | | | | |
| C | | | | | | | | |
| D | | | | HAD$ | | | | |
| E | | | HE* (7) | | | | THE$ | |
| F | FOR$ | | | | | | | |
| G | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | |
| I | I* (1) | | HIS$ | | WITH$ | | | |
| J | | | | | | | | |
| K | | | | | | | | |
| L | | | | | | | | |
| M | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | |
| O | ON$ | | | | | | | |
| P | | | | | | | | |
| Q | | | | | | | | |
| R | | | | | | | | HER$ |
| S | | IS$ | | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | |
| U | | | | | | | | |
| V | | | | HAVE$ | | | | |
| W | W* (4) | | | | | | | |
| X | | | | | | | | |
| Y | | | | | | | | |
| Z | | | | | | | | |

# Trie for 31 Most Common English Words

`add("WAS")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ |
| A | A* (5) | | HA* (3) | | | | THAT$ | |
| B | BUT$ | | | | | | | |
| C | | | | | | | | |
| D | | | | HAD$ | | | | |
| E | | | HE* (7) | | | | THE$ | |
| F | FOR$ | | | | | | | |
| G | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | |
| I | I* (1) | | HIS$ | | WITH$ | | | |
| J | | | | | | | | |
| K | | | | | | | | |
| L | | | | | | | | |
| M | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | |
| O | ON$ | | | | | | | |
| P | | | | | | | | |
| Q | | | | | | | | |
| R | | | | | | | | HER$ |
| S | | IS$ | | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | |
| U | | | | | | | | |
| V | | | | HAVE$ | | | | |
| W | W* (4) | | | | | | | |
| X | | | | | | | | |
| Y | | | | | | | | |
| Z | | | | | | | | |

# Trie for 31 Most Common English Words

`add("WAS")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | |
| B | BUT$ | | | | | | | |
| C | | | | | | | | |
| D | | | | HAD$ | | | | |
| E | | | HE* (7) | | | | THE$ | |
| F | FOR$ | | | | | | | |
| G | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | |
| I | I* (1) | | HIS$ | | WITH$ | | | |
| J | | | | | | | | |
| K | | | | | | | | |
| L | | | | | | | | |
| M | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | |
| O | ON$ | | | | | | | |
| P | | | | | | | | |
| Q | | | | | | | | |
| R | | | | | | | | HER$ |
| S | | IS$ | | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | |
| U | | | | | | | | |
| V | | | | HAVE$ | | | | |
| W | W* (4) | | | | | | | |
| X | | | | | | | | |
| Y | | | | | | | | |
| Z | | | | | | | | |

# Trie for 31 Most Common English Words

`add("ARE")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | |
| B | BUT$ | | | | | | | |
| C | | | | | | | | |
| D | | | | HAD$ | | | | |
| E | | | HE* (7) | | | | THE$ | |
| F | FOR$ | | | | | | | |
| G | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | |
| I | I* (1) | | HIS$ | | WITH$ | | | |
| J | | | | | | | | |
| K | | | | | | | | |
| L | | | | | | | | |
| M | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | |
| O | ON$ | | | | | | | |
| P | | | | | | | | |
| Q | | | | | | | | |
| R | | | | | | | | HER$ |
| S | | IS$ | | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | |
| U | | | | | | | | |
| V | | | | HAVE$ | | | | |
| W | W* (4) | | | | | | | |
| X | | | | | | | | |
| Y | | | | | | | | |
| Z | | | | | | | | |

# Trie for 31 Most Common English Words

`add("ARE")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | |
| B | BUT$ | | | | | | | |
| C | | | | | | | | |
| D | | | | HAD$ | | | | |
| E | | | HE* (7) | | | | THE$ | |
| F | FOR$ | | | | | | | |
| G | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | |
| I | I* (1) | | HIS$ | | WITH$ | | | |
| J | | | | | | | | |
| K | | | | | | | | |
| L | | | | | | | | |
| M | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | |
| O | ON$ | | | | | | | |
| P | | | | | | | | |
| Q | | | | | | | | |
| R | | | | | | ARE$ | | HER$ |
| S | | IS$ | | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | |
| U | | | | | | | | |
| V | | | | HAVE$ | | | | |
| W | W* (4) | | | | | | | |
| X | | | | | | | | |
| Y | | | | | | | | |
| Z | | | | | | | | |

# Trie for 31 Most Common English Words

`add("BY")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | |
| B | BUT$ | | | | | | | |
| C | | | | | | | | |
| D | | | | HAD$ | | | | |
| E | | | HE* (7) | | | | THE$ | |
| F | FOR$ | | | | | | | |
| G | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | |
| I | I* (1) | | HIS$ | | WITH$ | | | |
| J | | | | | | | | |
| K | | | | | | | | |
| L | | | | | | | | |
| M | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | |
| O | ON$ | | | | | | | |
| P | | | | | | | | |
| Q | | | | | | | | |
| R | | | | | | ARE$ | | HER$ |
| S | | IS$ | | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | |
| U | | | | | | | | |
| V | | | | HAVE$ | | | | |
| W | W* (4) | | | | | | | |
| X | | | | | | | | |
| Y | | | | | | | | |
| Z | | | | | | | | |

# Trie for 31 Most Common English Words

`add("BY")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | |
| B | B* (8) | | | | | | | | |
| C | | | | | | | | | |
| D | | | | HAD$ | | | | | |
| E | | | HE* (7) | | | | THE$ | | |
| F | FOR$ | | | | | | | | |
| G | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | | | |
| J | | | | | | | | | |
| K | | | | | | | | | |
| L | | | | | | | | | |
| M | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | |
| O | ON$ | | | | | | | | |
| P | | | | | | | | | |
| Q | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | |
| S | | IS$ | | | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | |
| U | | | | | | | | | BUT$ |
| V | | | | HAVE$ | | | | | |
| W | W* (4) | | | | | | | | |
| X | | | | | | | | | |
| Y | | | | | | | | | BY$ |
| Z | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("OR")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | |
| B | B* (8) | | | | | | | | |
| C | | | | | | | | | |
| D | | | | HAD$ | | | | | |
| E | | | HE* (7) | | | | THE$ | | |
| F | FOR$ | | | | | | | | |
| G | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | | | |
| J | | | | | | | | | |
| K | | | | | | | | | |
| L | | | | | | | | | |
| M | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | |
| O | ON$ | | | | | | | | |
| P | | | | | | | | | |
| Q | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | |
| S | | IS$ | | | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | |
| U | | | | | | | | | BUT$ |
| V | | | | HAVE$ | | | | | |
| W | W* (4) | | | | | | | | |
| X | | | | | | | | | |
| Y | | | | | | | | | BY$ |
| Z | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("OR")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | | |
| B | B* (8) | | | | | | | | | |
| C | | | | | | | | | | |
| D | | | | HAD$ | | | | | | |
| E | | | HE* (7) | | | | THE$ | | | |
| F | FOR$ | | | | | | | | | |
| G | | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | | | | |
| J | | | | | | | | | | |
| K | | | | | | | | | | |
| L | | | | | | | | | | |
| M | | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | | ON$ |
| O | O* (9) | | | | | | | | | |
| P | | | | | | | | | | |
| Q | | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | | OR$ |
| S | | IS$ | | | | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | | |
| U | | | | | | | | | BUT$ | |
| V | | | | HAVE$ | | | | | | |
| W | W* (4) | | | | | | | | | |
| X | | | | | | | | | | |
| Y | | | | | | | | | BY$ | |
| Z | | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("AS")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | | |
| B | B* (8) | | | | | | | | | |
| C | | | | | | | | | | |
| D | | | | HAD$ | | | | | | |
| E | | | HE* (7) | | | | THE$ | | | |
| F | FOR$ | | | | | | | | | |
| G | | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | | | | |
| J | | | | | | | | | | |
| K | | | | | | | | | | |
| L | | | | | | | | | | |
| M | | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | | ON$ |
| O | O* (9) | | | | | | | | | |
| P | | | | | | | | | | |
| Q | | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | | OR$ |
| S | | IS$ | | | | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | | |
| U | | | | | | | | | BUT$ | |
| V | | | | HAVE$ | | | | | | |
| W | W* (4) | | | | | | | | | |
| X | | | | | | | | | | |
| Y | | | | | | | | | BY$ | |
| Z | | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("AS")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | | |
| B | B* (8) | | | | | | | | | |
| C | | | | | | | | | | |
| D | | | | HAD$ | | | | | | |
| E | | | HE* (7) | | | | THE$ | | | |
| F | FOR$ | | | | | | | | | |
| G | | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | | | | |
| J | | | | | | | | | | |
| K | | | | | | | | | | |
| L | | | | | | | | | | |
| M | | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | | ON$ |
| O | O* (9) | | | | | | | | | |
| P | | | | | | | | | | |
| Q | | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | | OR$ |
| S | | IS$ | | | | AS$ | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | | |
| U | | | | | | | | | BUT$ | |
| V | | | | HAVE$ | | | | | | |
| W | W* (4) | | | | | | | | | |
| X | | | | | | | | | | |
| Y | | | | | | | | | BY$ | |
| Z | | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("THIS")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | | |
| B | B* (8) | | | | | | | | | |
| C | | | | | | | | | | |
| D | | | | HAD$ | | | | | | |
| E | | | HE* (7) | | | | THE$ | | | |
| F | FOR$ | | | | | | | | | |
| G | | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | | | | |
| J | | | | | | | | | | |
| K | | | | | | | | | | |
| L | | | | | | | | | | |
| M | | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | | ON$ |
| O | O* (9) | | | | | | | | | |
| P | | | | | | | | | | |
| Q | | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | | OR$ |
| S | | IS$ | | | | AS$ | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | | |
| U | | | | | | | | | BUT$ | |
| V | | | | HAVE$ | | | | | | |
| W | W* (4) | | | | | | | | | |
| X | | | | | | | | | | |
| Y | | | | | | | | | BY$ | |
| Z | | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("THIS")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | | |
| B | B* (8) | | | | | | | | | |
| C | | | | | | | | | | |
| D | | | | HAD$ | | | | | | |
| E | | | HE* (7) | | | | THE$ | | | |
| F | FOR$ | | | | | | | | | |
| G | | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | THIS$ | | | |
| J | | | | | | | | | | |
| K | | | | | | | | | | |
| L | | | | | | | | | | |
| M | | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | | ON$ |
| O | O* (9) | | | | | | | | | |
| P | | | | | | | | | | |
| Q | | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | | OR$ |
| S | | IS$ | | | | AS$ | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | | |
| U | | | | | | | | | BUT$ | |
| V | | | | HAVE$ | | | | | | |
| W | W* (4) | | | | | | | | | |
| X | | | | | | | | | | |
| Y | | | | | | | | | BY$ | |
| Z | | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("YOU")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | | |
| B | B* (8) | | | | | | | | | |
| C | | | | | | | | | | |
| D | | | | HAD$ | | | | | | |
| E | | | HE* (7) | | | | THE$ | | | |
| F | FOR$ | | | | | | | | | |
| G | | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | THIS$ | | | |
| J | | | | | | | | | | |
| K | | | | | | | | | | |
| L | | | | | | | | | | |
| M | | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | | ON$ |
| O | O* (9) | | | | | | | | | |
| P | | | | | | | | | | |
| Q | | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | | OR$ |
| S | | IS$ | | | | AS$ | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | | |
| U | | | | | | | | | BUT$ | |
| V | | | | HAVE$ | | | | | | |
| W | W* (4) | | | | | | | | | |
| X | | | | | | | | | | |
| Y | | | | | | | | | BY$ | |
| Z | | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("YOU")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | | |
| B | B* (8) | | | | | | | | | |
| C | | | | | | | | | | |
| D | | | | HAD$ | | | | | | |
| E | | | HE* (7) | | | | THE$ | | | |
| F | FOR$ | | | | | | | | | |
| G | | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | THIS$ | | | |
| J | | | | | | | | | | |
| K | | | | | | | | | | |
| L | | | | | | | | | | |
| M | | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | | ON$ |
| O | O* (9) | | | | | | | | | |
| P | | | | | | | | | | |
| Q | | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | | OR$ |
| S | | IS$ | | | | AS$ | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | | |
| U | | | | | | | | | BUT$ | |
| V | | | | HAVE$ | | | | | | |
| W | W* (4) | | | | | | | | | |
| X | | | | | | | | | | |
| Y | YOU$ | | | | | | | | BY$ | |
| Z | | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("BE")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | | |
| B | B* (8) | | | | | | | | | |
| C | | | | | | | | | | |
| D | | | | HAD$ | | | | | | |
| E | | | HE* (7) | | | | THE$ | | | |
| F | FOR$ | | | | | | | | | |
| G | | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | THIS$ | | | |
| J | | | | | | | | | | |
| K | | | | | | | | | | |
| L | | | | | | | | | | |
| M | | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | | ON$ |
| O | O* (9) | | | | | | | | | |
| P | | | | | | | | | | |
| Q | | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | | OR$ |
| S | | IS$ | | | | AS$ | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | | |
| U | | | | | | | | | BUT$ | |
| V | | | | HAVE$ | | | | | | |
| W | W* (4) | | | | | | | | | |
| X | | | | | | | | | | |
| Y | YOU$ | | | | | | | | BY$ | |
| Z | | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("BE")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | | |
| B | B* (8) | | | | | | | | | |
| C | | | | | | | | | | |
| D | | | | HAD$ | | | | | | |
| E | | | HE* (7) | | | | THE$ | | BE$ | |
| F | FOR$ | | | | | | | | | |
| G | | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | THIS$ | | | |
| J | | | | | | | | | | |
| K | | | | | | | | | | |
| L | | | | | | | | | | |
| M | | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | | ON$ |
| O | O* (9) | | | | | | | | | |
| P | | | | | | | | | | |
| Q | | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | | OR$ |
| S | | IS$ | | | | AS$ | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | | |
| U | | | | | | | | | BUT$ | |
| V | | | | HAVE$ | | | | | | |
| W | W* (4) | | | | | | | | | |
| X | | | | | | | | | | |
| Y | YOU$ | | | | | | | | BY$ | |
| Z | | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("FROM")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | | |
| B | B* (8) | | | | | | | | | |
| C | | | | | | | | | | |
| D | | | | HAD$ | | | | | | |
| E | | | HE* (7) | | | | THE$ | | BE$ | |
| F | FOR$ | | | | | | | | | |
| G | | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | THIS$ | | | |
| J | | | | | | | | | | |
| K | | | | | | | | | | |
| L | | | | | | | | | | |
| M | | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | | ON$ |
| O | O* (9) | | | | | | | | | |
| P | | | | | | | | | | |
| Q | | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | | OR$ |
| S | | IS$ | | | | AS$ | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | | |
| U | | | | | | | | | BUT$ | |
| V | | | | HAVE$ | | | | | | |
| W | W* (4) | | | | | | | | | |
| X | | | | | | | | | | |
| Y | YOU$ | | | | | | | | BY$ | |
| Z | | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("FROM")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | | | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | | | |
| B | B* (8) | | | | | | | | | | |
| C | | | | | | | | | | | |
| D | | | | HAD$ | | | | | | | |
| E | | | HE* (7) | | | | THE$ | | BE$ | | |
| F | F* (10) | | | | | | | | | | |
| G | | | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | THIS$ | | | | |
| J | | | | | | | | | | | |
| K | | | | | | | | | | | |
| L | | | | | | | | | | | |
| M | | | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | | ON$ | |
| O | O* (9) | | | | | | | | | | FOR$ |
| P | | | | | | | | | | | |
| Q | | | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | | OR$ | FROM$ |
| S | | IS$ | | | | AS$ | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | | | |
| U | | | | | | | | | BUT$ | | |
| V | | | | HAVE$ | | | | | | | |
| W | W* (4) | | | | | | | | | | |
| X | | | | | | | | | | | |
| Y | YOU$ | | | | | | | | BY$ | | |
| Z | | | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("OF")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | | | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | | | |
| B | B* (8) | | | | | | | | | | |
| C | | | | | | | | | | | |
| D | | | | HAD$ | | | | | | | |
| E | | | HE* (7) | | | | THE$ | | BE$ | | |
| F | F* (10) | | | | | | | | | | |
| G | | | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | THIS$ | | | | |
| J | | | | | | | | | | | |
| K | | | | | | | | | | | |
| L | | | | | | | | | | | |
| M | | | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | | ON$ | |
| O | O* (9) | | | | | | | | | | FOR$ |
| P | | | | | | | | | | | |
| Q | | | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | | OR$ | FROM$ |
| S | | IS$ | | | | AS$ | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | | | |
| U | | | | | | | | | BUT$ | | |
| V | | | | HAVE$ | | | | | | | |
| W | W* (4) | | | | | | | | | | |
| X | | | | | | | | | | | |
| Y | YOU$ | | | | | | | | BY$ | | |
| Z | | | | | | | | | | | |

# Trie for 31 Most Common English Words

`add("OF")`

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $ | | I$ | | | | A$ | | HE$ | | | |
| A | A* (5) | | HA* (3) | | WAS$ | | THAT$ | | | | |
| B | B* (8) | | | | | | | | | | |
| C | | | | | | | | | | | |
| D | | | | HAD$ | | | | | | | |
| E | | | HE* (7) | | | | THE$ | | BE$ | | |
| F | F* (10) | | | | | | | | | OF$ | |
| G | | | | | | | | | | | |
| H | H* (2) | | | | WHICH$ | | | | | | |
| I | I* (1) | | HIS$ | | WITH$ | | THIS$ | | | | |
| J | | | | | | | | | | | |
| K | | | | | | | | | | | |
| L | | | | | | | | | | | |
| M | | | | | | | | | | | |
| N | NOT$ | IN$ | | | | AND$ | | | | ON$ | |
| O | O* (9) | | | | | | | | | | FOR$ |
| P | | | | | | | | | | | |
| Q | | | | | | | | | | | |
| R | | | | | | ARE$ | | HER$ | | OR$ | FROM$ |
| S | | IS$ | | | | AS$ | | | | | |
| T | TH* (6) | IT$ | | | | AT$ | | | | | |
| U | | | | | | | | | BUT$ | | |
| V | | | | HAVE$ | | | | | | | |
| W | W* (4) | | | | | | | | | | |
| X | | | | | | | | | | | |
| Y | YOU$ | | | | | | | | BY$ | | |
| Z | | | | | | | | | | | |

# Disadvantage of Tries

- Table-based tries typically waste large amounts of memory
- Often table-based tries are used for the first few layers, while lower levels use a less memory intensive data structure
- These days memory is less of a problem so table-based tries are acceptable for some applications
- There are many implementations of tries each suited to a particular task
- For DNA tries are not too wasteful

# Disadvantage of Tries

- Table-based tries typically waste large amounts of memory
- Often table-based tries are used for the first few layers, while lower levels use a less memory intensive data structure
- These days memory is less of a problem so table-based tries are acceptable for some applications
- There are many implementations of tries each suited to a particular task
- For DNA tries are not too wasteful

# Disadvantage of Tries

- Table-based tries typically waste large amounts of memory
- Often table-based tries are used for the first few layers, while lower levels use a less memory intensive data structure
- These days memory is less of a problem so table-based tries are acceptable for some applications
- There are many implementations of tries each suited to a particular task
- For DNA tries are not too wasteful

# Disadvantage of Tries

- Table-based tries typically waste large amounts of memory
- Often table-based tries are used for the first few layers, while lower levels use a less memory intensive data structure
- These days memory is less of a problem so table-based tries are acceptable for some applications
- There are many implementations of tries each suited to a particular task
- For DNA tries are not too wasteful

# Disadvantage of Tries

- Table-based tries typically waste large amounts of memory
- Often table-based tries are used for the first few layers, while lower levels use a less memory intensive data structure
- These days memory is less of a problem so table-based tries are acceptable for some applications
- There are many implementations of tries each suited to a particular task
- For DNA tries are not too wasteful

# Disadvantage of Tries

- Table-based tries typically waste large amounts of memory
- Often table-based tries are used for the first few layers, while lower levels use a less memory intensive data structure
- These days memory is less of a problem so table-based tries are acceptable for some applications
- There are many implementations of tries each suited to a particular task
- For DNA tries are not too wasteful (although maybe for proteins)

# Outline

1. Reconstruction
2. Digital Trees
3. **Suffix Trees**
4. Repeats

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

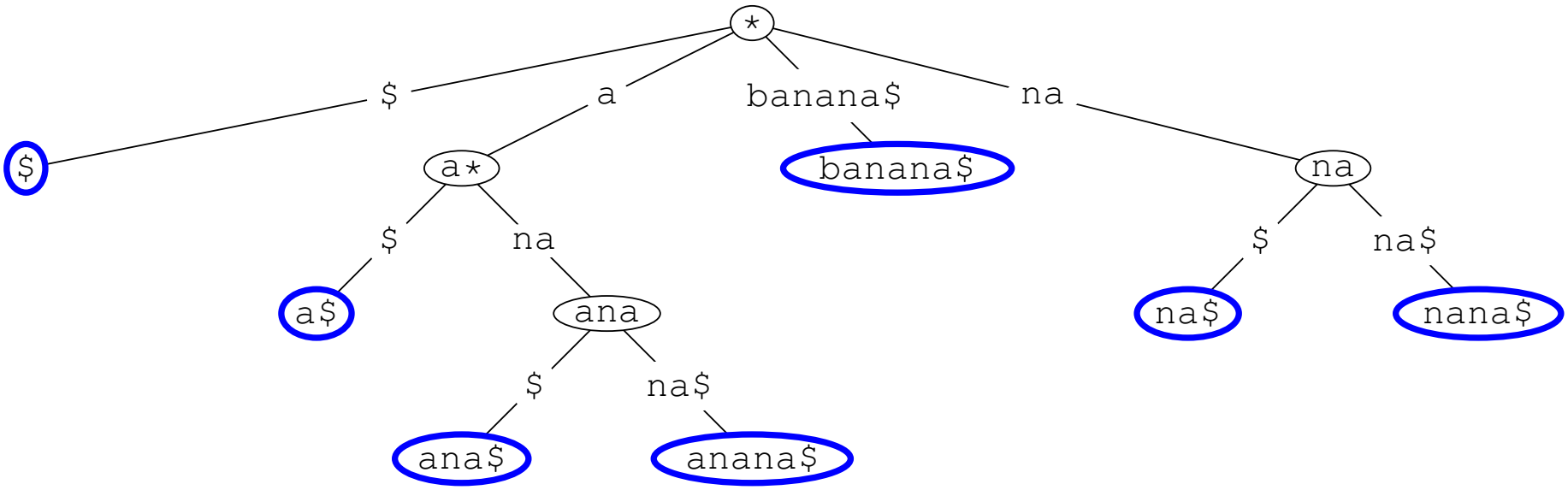
GGTTAAA CTCTACTCGGCATGT

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# Suffix Tree

- Suffix tree is a trie of all suffixes of a string
- E.g. banana

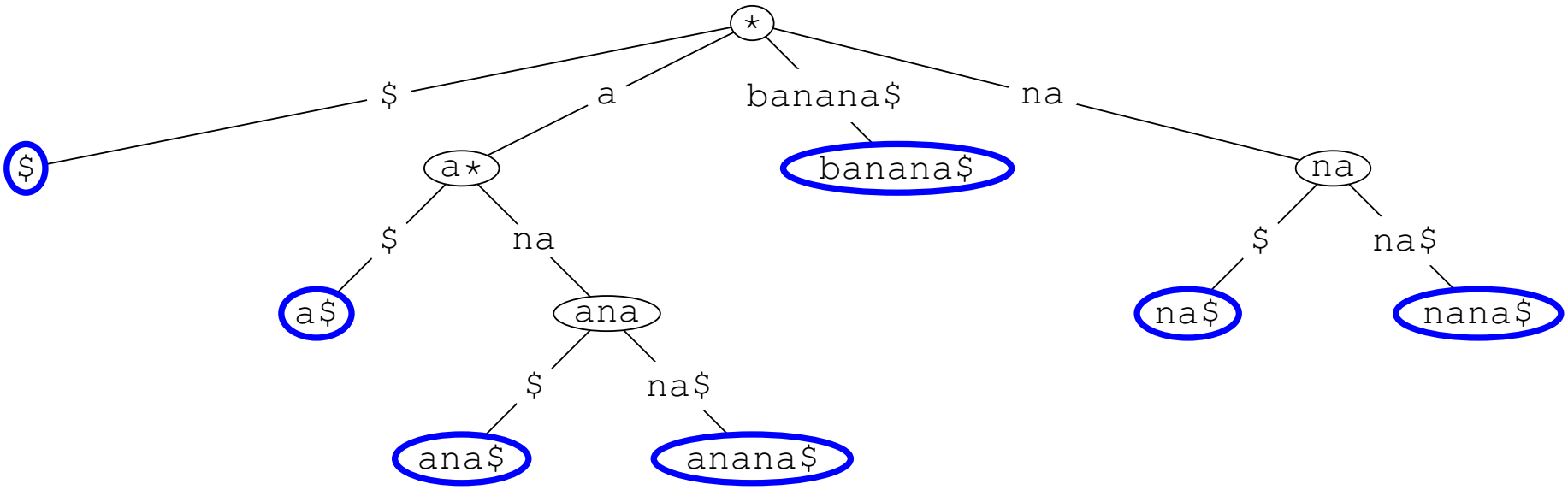
{$, a$, na$, ana$, nana$, anana$, banana$}



# Suffix Tree

- Suffix tree is a trie of all suffixes of a string
- E.g. banana

{$, a$, na$, ana$, nana$, anana$, banana$}



# Importance of Suffix Tree

- Fast algorithms to compute Suffix trees (and the related suffix array) are relative new (15-20 year old) data structures
- They are very important for string-based algorithms
- The classic application is in finding a match for a query string, $Q$, in a text, $T$

# Importance of Suffix Tree

- Fast algorithms to compute Suffix trees (and the related suffix array) are relative new (15-20 year old) data structures
- They are very important for string-based algorithms
- The classic application is in finding a match for a query string, $Q$, in a text, $T$

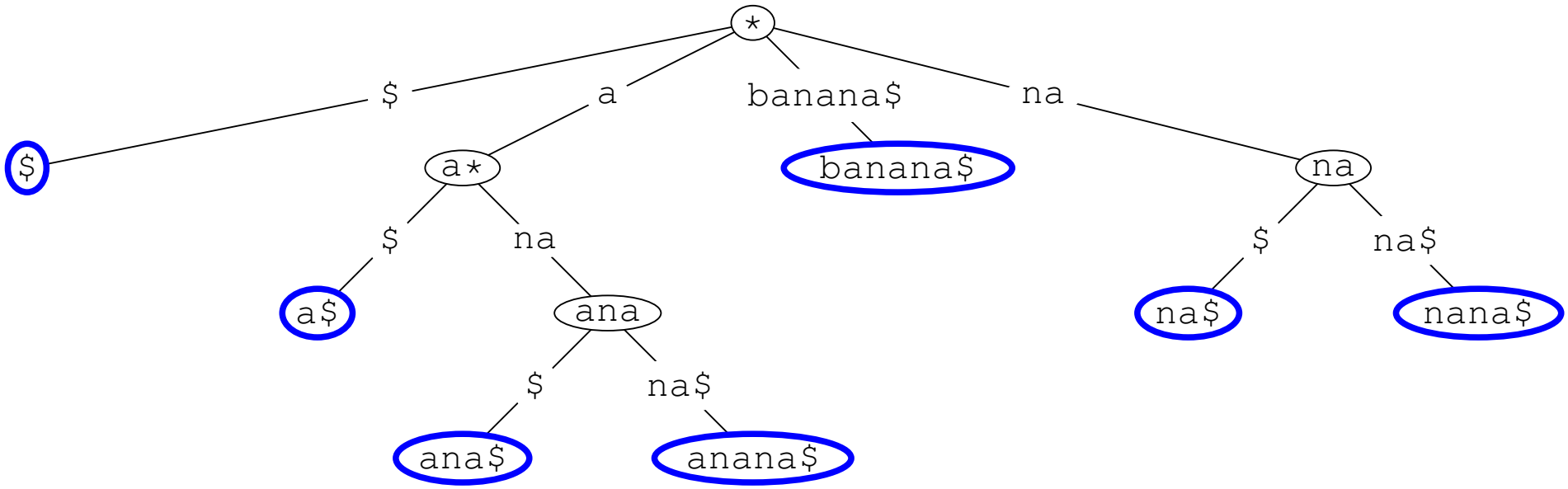
# Importance of Suffix Tree

- Fast algorithms to compute Suffix trees (and the related suffix array) are relative new (15-20 year old) data structures
- They are very important for string-based algorithms
- The classic application is in finding a match for a query string, $Q$, in a text, $T$

# String Matching

- To find a match of a query string, $Q = an$, in a text, $T = banana$, we can first construct the suffix tree of the string $T$ we then simple look up the query, $Q$, using the trie

{$, a$, na$, ana$, nana$, anana$, banana$}



# Complexity of Suffix Trees

- Using a regular trie for a suffix tree would typically use far too much memory to be useful
- However, by using pointers to the original text it is possible to build a suffix tree using $O(n)$ memory where $n$ is the length of the text
- Furthermore (and rather incredibly) there is a linear time ($O(n)$) algorithm to construct the trie
- The algorithm is not however trivial to understand

# Complexity of Suffix Trees

- Using a regular trie for a suffix tree would typically use far too much memory to be useful
- However, by using pointers to the original text it is possible to build a suffix tree using $O(n)$ memory where $n$ is the length of the text
- Furthermore (and rather incredibly) there is a linear time ($O(n)$) algorithm to construct the trie
- The algorithm is not however trivial to understand

# Complexity of Suffix Trees

- Using a regular trie for a suffix tree would typically use far too much memory to be useful
- However, by using pointers to the original text it is possible to build a suffix tree using $O(n)$ memory where $n$ is the length of the text
- Furthermore (and rather incredibly) there is a linear time ($O(n)$) algorithm to construct the trie
- The algorithm is not however trivial to understand

# Complexity of Suffix Trees

- Using a regular trie for a suffix tree would typically use far too much memory to be useful
- However, by using pointers to the original text it is possible to build a suffix tree using $O(n)$ memory where $n$ is the length of the text
- Furthermore (and rather incredibly) there is a linear time ($O(n)$) algorithm to construct the trie
- The algorithm is not however trivial to understand

# Uses of Suffix Trees

- Suffix trees are efficient whenever it is likely that you will do multiple searches
- Exact word matching is in itself a very important application
- Suffix trees in combination with dynamic programming (which we look at next) can be used to do inexact matching (finding the match with the smallest edit distance)
- Suffix trees get used in bioinformatics, advanced machine learning algorithms, . . .
- In case you are interested there are also prefix trees containing all prefixes

# Uses of Suffix Trees

- Suffix trees are efficient whenever it is likely that you will do multiple searches
- Exact word matching is in itself a very important application
- Suffix trees in combination with dynamic programming (which we look at next) can be used to do inexact matching (finding the match with the smallest edit distance)
- Suffix trees get used in bioinformatics, advanced machine learning algorithms, . . .
- In case you are interested there are also prefix trees containing all prefixes

# Uses of Suffix Trees

- Suffix trees are efficient whenever it is likely that you will do multiple searches
- Exact word matching is in itself a very important application
- Suffix trees in combination with dynamic programming (which we look at next) can be used to do inexact matching (finding the match with the smallest edit distance)
- Suffix trees get used in bioinformatics, advanced machine learning algorithms, . . .
- In case you are interested there are also prefix trees containing all prefixes

# Uses of Suffix Trees

- Suffix trees are efficient whenever it is likely that you will do multiple searches
- Exact word matching is in itself a very important application
- Suffix trees in combination with dynamic programming (which we look at next) can be used to do inexact matching (finding the match with the smallest edit distance)
- Suffix trees get used in bioinformatics, advanced machine learning algorithms, . . .
- In case you are interested there are also prefix trees containing all prefixes

# Uses of Suffix Trees

- Suffix trees are efficient whenever it is likely that you will do multiple searches
- Exact word matching is in itself a very important application
- Suffix trees in combination with dynamic programming (which we look at next) can be used to do inexact matching (finding the match with the smallest edit distance)
- Suffix trees get used in bioinformatics, advanced machine learning algorithms, . . .
- In case you are interested there are also prefix trees containing all prefixes

# Outline

1. Reconstruction
2. Digital Trees
3. Suffix Trees
4. **Repeats**

GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG
GGTTAAATCCTCTACTCGGCATGTCCCGTCTCGAGCAACG

GGTTAAA CTCTACTCGGCATGT

| | | | | | |
|---|---|---|---|---|---|
| GAGCA | GGTTA | CTACT | GTTAA | CATGT | TCGGC |
| TACTC | TTAAA | CCGTC | GGTTA | GGTTA | ACTCG |
| GGCAT | CATGT | CTACT | GTTAA | TTAAA | TGTCC |
| GGCAT | GAGCA | TCTCG | AGCAA | GCATG | CTCTA |
| AGCAA | CGTCT | TCCCG | CTCGA | CGAGC | CAACG |

# New Generation Sequencers

- The estimated cost of sequencing the human genome in 2005 was $\$10\,000\,000$
- To reduce the cost there was and is a drive to produce new sequencing machines
- These tend to read much shorter sections of DNA (e.g. 20-100nt)
- Can these be re-assembled?

# New Generation Sequencers

- The estimated cost of sequencing the human genome in 2005 was $\$10\,000\,000$
- To reduce the cost there was and is a drive to produce new sequencing machines
- These tend to read much shorter sections of DNA (e.g. 20-100nt)
- Can these be re-assembled?

# New Generation Sequencers

- The estimated cost of sequencing the human genome in 2005 was $\$10\,000\,000$
- To reduce the cost there was and is a drive to produce new sequencing machines
- These tend to read much shorter sections of DNA (e.g. 20-100nt)
- Can these be re-assembled?

# New Generation Sequencers

- The estimated cost of sequencing the human genome in 2005 was $\$10\,000\,000$
- To reduce the cost there was and is a drive to produce new sequencing machines
- These tend to read much shorter sections of DNA (e.g. 20-100nt)
- Can these be re-assembled?

# Repeats

- The difficulty of assembly is caused by repeats

ATA$\boxed{\text{CCACCAT}}$GCCTCCTTGCTCCAAT$\boxed{\text{CCACCAT}}$CAAGGCG

- How many repeats are there in the human genome? (Incidentally the human genome is 3.2 billion base **pairs**)

- You can read DNA in either directions meaning there are effectively 6.4 billion bases

- This is an important question for developing new sequencing technologies

# Repeats

- The difficulty of assembly is caused by repeats

ATA$\boxed{\text{CCACCAT}}$GCCTCCTTGCTCCAAT$\boxed{\text{CCACCAT}}$CAAGGCG

- How many repeats are there in the human genome? (Incidentally the human genome is 3.2 billion base **pairs**)

- You can read DNA in either directions meaning there are effectively 6.4 billion bases

- This is an important question for developing new sequencing technologies

# Repeats

- The difficulty of assembly is caused by repeats

ATA$\boxed{\text{CCACCAT}}$GCCTCCTTGCTCCAAT$\boxed{\text{CCACCAT}}$CAAGGCG

- How many repeats are there in the human genome? (Incidentally the human genome is 3.2 billion base **pairs**)

- You can read DNA in either directions meaning there are effectively 6.4 billion bases

- This is an important question for developing new sequencing technologies

# Repeats

- The difficulty of assembly is caused by repeats

ATA$\boxed{\text{CCACCAT}}$GCCTCCTTGCTCCAAT$\boxed{\text{CCACCAT}}$CAAGGCG

- How many repeats are there in the human genome? (Incidentally the human genome is 3.2 billion base **pairs**)

- You can read DNA in either directions meaning there are effectively 6.4 billion bases

- This is an important question for developing new sequencing technologies

# Repeats

- The difficulty of assembly is caused by repeats

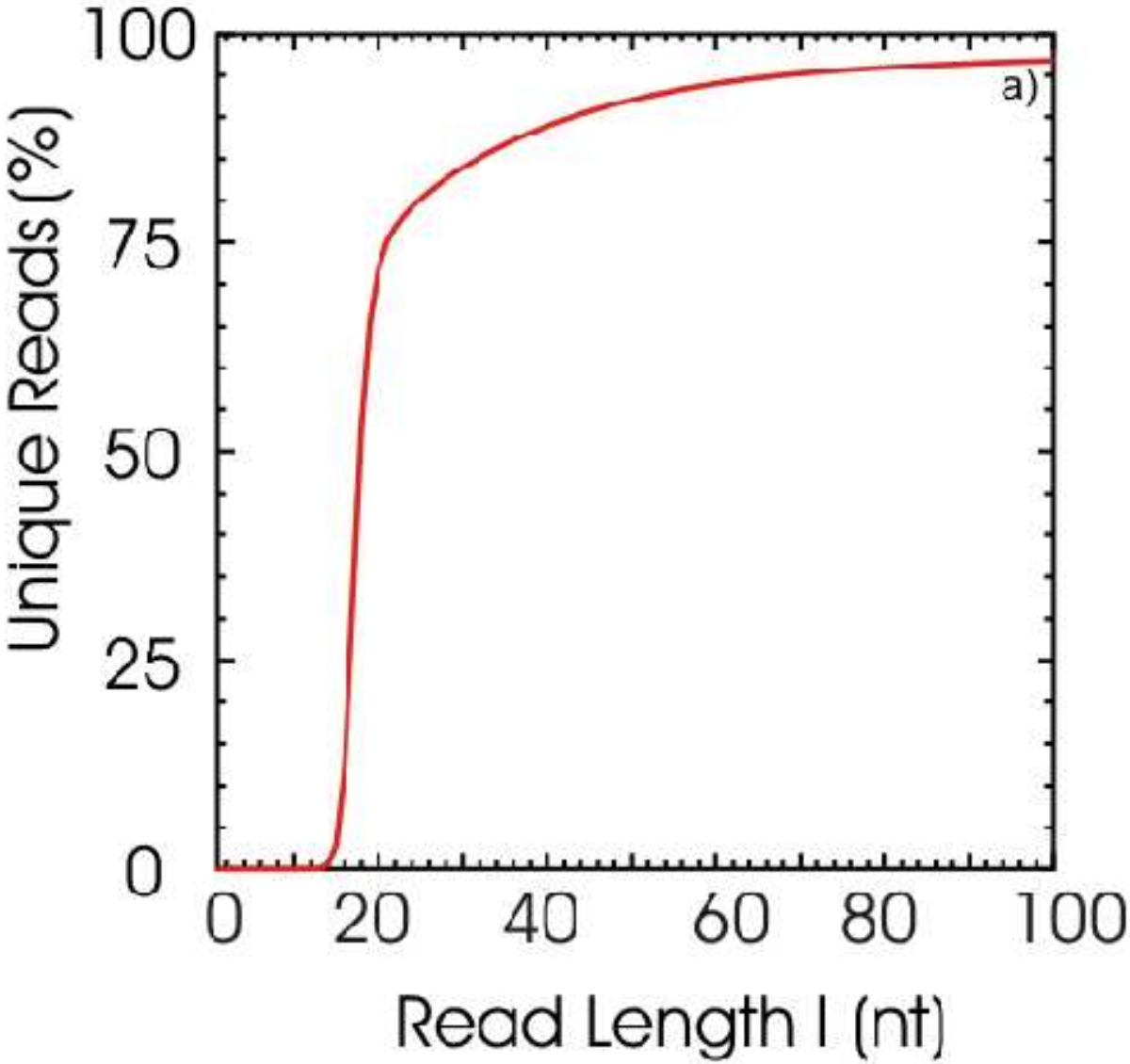
ATA$\boxed{\text{CCACCAT}}$GCCTCCTTGCTCCAAT$\boxed{\text{CCACCAT}}$CAAGGCG

- How many repeats are there in the human genome? (Incidentally the human genome is 3.2 billion base **pairs**)

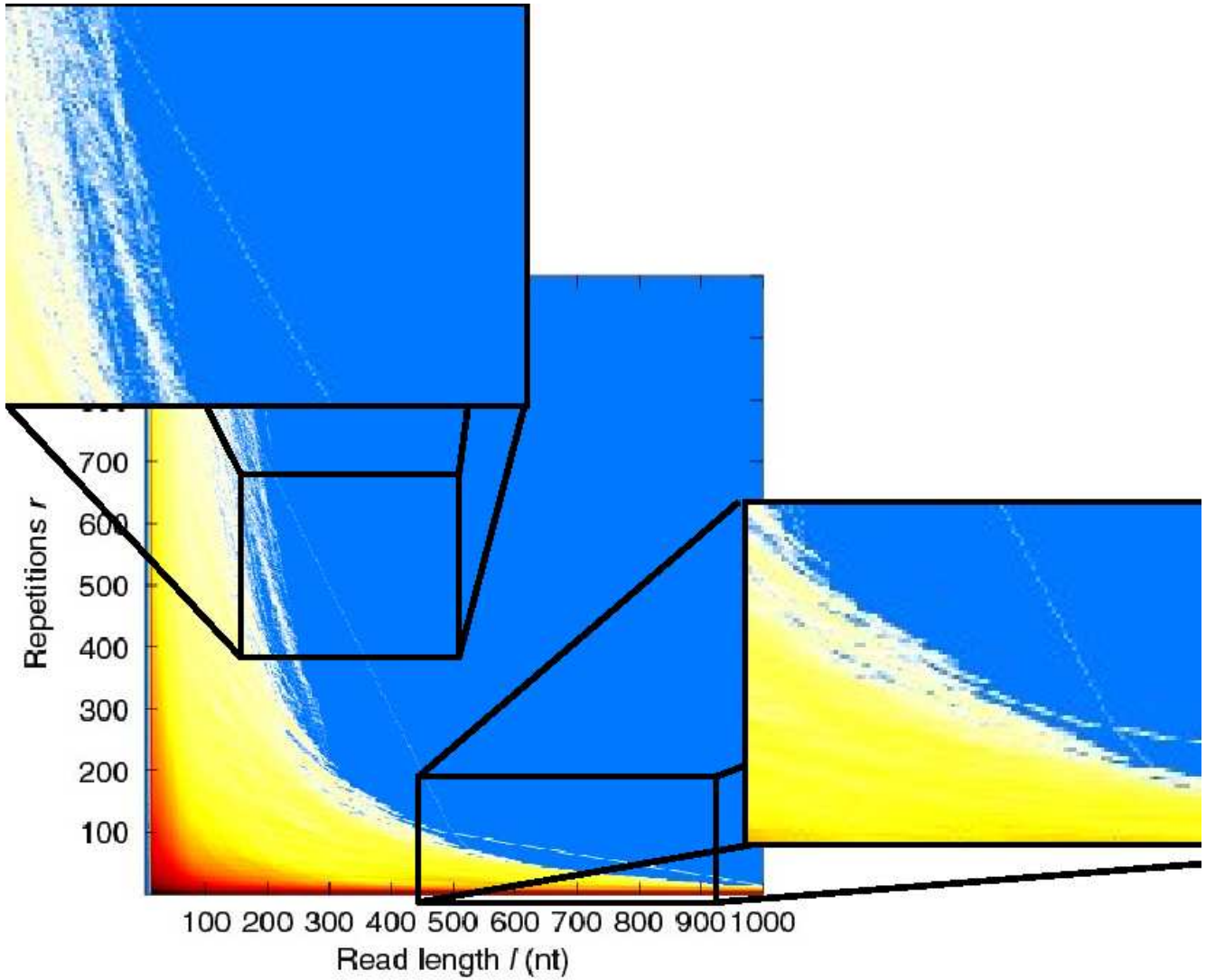
- You can read DNA in either directions meaning there are effectively 6.4 billion bases

- This is an important question for developing new sequencing technologies

# Repeats in Human Genome



# Repeats Structure



# Computing Repeats

- A naive program would take $n^2$ operations where $n = 6.4 \times 10^9$
- If we used this we would still be waiting for the program to finish
- Could not answer this question a few years ago—not because computers weren't powerful, but because the algorithms had not been developed
- However using suffix arrays counting repeats was made easy

# Computing Repeats

- A naive program would take $n^2$ operations where $n = 6.4 \times 10^9$
- If we used this we would still be waiting for the program to finish
- Could not answer this question a few years ago—not because computers weren't powerful, but because the algorithms had not been developed
- However using suffix arrays counting repeats was made easy

# Computing Repeats

- A naive program would take $n^2$ operations where $n = 6.4 \times 10^9$
- If we used this we would still be waiting for the program to finish
- Could not answer this question a few years ago—not because computers weren't powerful, but because the algorithms had not been developed
- However using suffix arrays counting repeats was made easy

# Computing Repeats

- A naive program would take $n^2$ operations where $n = 6.4 \times 10^9$
- If we used this we would still be waiting for the program to finish
- Could not answer this question a few years ago—not because computers weren't powerful, but because the algorithms had not been developed
- However using suffix arrays counting repeats was made easy

# Computing Repeats

- A naive program would take $n^2$ operations where $n = 6.4 \times 10^9$
- If we used this we would still be waiting for the program to finish
- Could not answer this question a few years ago—not because computers weren't powerful, but because the algorithms had not been developed
- However using suffix arrays counting repeats was made easy

# Summary

- The incredibly swift development in algorithms has leveraged the advances in computer power and smart algorithms
- Digital trees or tries, though simple can significantly speed up algorithms for re-assembly
- Suffix trees are conceptually simple (digital trees of all suffixes)
- They can vastly speed up operations such as matching
- What is complicated is the algorithm to construct them efficiently (though today they are widely available in programming languages such as python)

# Summary

- The incredibly swift development in algorithms has leveraged the advances in computer power and smart algorithms
- Digital trees or tries, though simple can significantly speed up algorithms for re-assembly
- Suffix trees are conceptually simple (digital trees of all suffixes)
- They can vastly speed up operations such as matching
- What is complicated is the algorithm to construct them efficiently (though today they are widely available in programming languages such as python)

# Summary

- The incredibly swift development in algorithms has leveraged the advances in computer power and smart algorithms
- Digital trees or tries, though simple can significantly speed up algorithms for re-assembly
- Suffix trees are conceptually simple (digital trees of all suffixes)
- They can vastly speed up operations such as matching
- What is complicated is the algorithm to construct them efficiently (though today they are widely available in programming languages such as python)

# Summary

- The incredibly swift development in algorithms has leveraged the advances in computer power and smart algorithms
- Digital trees or tries, though simple can significantly speed up algorithms for re-assembly
- Suffix trees are conceptually simple (digital trees of all suffixes)
- They can vastly speed up operations such as matching
- What is complicated is the algorithm to construct them efficiently (though today they are widely available in programming languages such as python)

# Summary

- The incredibly swift development in algorithms has leveraged the advances in computer power and smart algorithms
- Digital trees or tries, though simple can significantly speed up algorithms for re-assembly
- Suffix trees are conceptually simple (digital trees of all suffixes)
- They can vastly speed up operations such as matching
- What is complicated is the algorithm to construct them efficiently (though today they are widely available in programming languages such as python)

# Summary

- The incredibly swift development in algorithms has leveraged the advances in computer power and smart algorithms
- Digital trees or tries, though simple can significantly speed up algorithms for re-assembly
- Suffix trees are conceptually simple (digital trees of all suffixes)
- They can vastly speed up operations such as matching
- What is complicated is the algorithm to construct them efficiently (though today they are widely available in programming languages such as python)